

SQL untuk Big Data: DuckDB dan Format Data

Pertemuan 3 — Analisis Big Data

S1 Teknik Informatika / Sistem Informasi | Semester Ganjil 2026

Agenda Pertemuan 3

1. Mengapa DuckDB? OLAP in-process tanpa server
2. Query langsung ke file: Parquet, CSV, JSON
3. Predicate Pushdown dan Column Pruning
4. Window Functions SQL: RANK, SUM OVER, LAG/LEAD
5. Partitioned Parquet untuk query cepat
6. Interoperabilitas DuckDB <-> Polars via Apache Arrow
7. Perbandingan format data: CSV vs Parquet vs Arrow IPC

Check-in Milestone 1 Tugas 1 hari ini!

Mengapa DuckDB?

DuckDB vs Database OLTP Tradisional

Aspek	MySQL/Postgres (OLTP)	DuckDB (OLAP)
Target beban kerja	Transaksi baris-per-baris	Analitik agregasi kolom
Server terpisah	Wajib dikonfigurasi	Tidak perlu (in-process)
Penyimpanan	Row-oriented	Columnar
Query file langsung	Tidak	Ya (Parquet, CSV, JSON)
Setup	Instalasi server + user	<code>pip install duckdb</code>

Konsep Kunci

- DuckDB berjalan **in-process**, tertanam di aplikasi Python seperti library biasa.
- Dioptimalkan untuk **query analitik**: `GROUP BY`, `JOIN`, agregasi pada jutaan baris.
- Mendukung **zero-copy** interoperabilitas dengan Apache Arrow dan Polars.

Query Langsung ke File

DuckDB dapat meng-query file Parquet, CSV, dan JSON **tanpa** proses import data terlebih dahulu.

```
import duckdb

con = duckdb.connect()

# Query langsung ke Parquet
con.execute("""
    SELECT jenis_layanan, COUNT(*) AS jumlah, AVG(durasi_menit) AS rata_durasi
    FROM 'layanannya.parquet'
    WHERE status = 'selesai'
    GROUP BY jenis_layanan
    ORDER BY jumlah DESC
""").df()

# Query langsung ke CSV dan JSON dengan sintaks yang sama
con.execute("SELECT * FROM 'layanannya.csv' LIMIT 5").df()
con.execute("SELECT * FROM 'layanannya_sample.json' LIMIT 5").df()
```

File menjadi "tabel virtual". DuckDB membaca hanya bagian yang diperlukan oleh query.

Kuis Checkpoint 1 (Menit 25)

Buka Wayground — Instructor-Paced Mode

3 soal konseptual OLAP dan DuckDB:

1. DuckDB paling tepat dikategorikan sebagai...
2. Keunggulan DuckDB saat melakukan query langsung pada file Parquet adalah...
3. Istilah zero-copy dalam pertukaran data antara Polars, DuckDB, dan Arrow berarti...

Waktu: 3 menit

Predicate Pushdown dan Column Pruning

Predicate Pushdown

```
# Filter diturunkan ke pembaca Parquet
con.execute("""
    SELECT * FROM 'layanan.parquet'
    WHERE provinsi = 'DKI Jakarta'
""").pl()
```

Hanya kolom yang disebut di `SELECT` yang dibaca dari disk.

Filter `WHERE` diterapkan **saat membaca file**, bukan setelah seluruh data dimuat.

Column Pruning (Projection Pushdown)

```
# Hanya 2 dari 9 kolom yang dibaca
con.execute("""
    SELECT jenis_layanan, biaya
    FROM 'layanan.parquet'
""").pl()
```

Kombinasi keduanya membuat DuckDB jauh lebih cepat dibanding membaca seluruh file lalu memfilter secara manual.

Window Functions SQL

Window functions menghasilkan agregasi **tanpa mereduksi jumlah baris**, berbeda dengan `GROUP BY`.

```
WITH ringkasan AS (  
    SELECT provinsi, jenis_layanan, COUNT(*) AS jumlah,  
           AVG(kepuasan) AS rata_kepuasan  
    FROM 'layanannya.parquet'  
    WHERE status = 'selesai'  
    GROUP BY provinsi, jenis_layanan  
)  
SELECT  
    provinsi, jenis_layanan, jumlah,  
    RANK() OVER (PARTITION BY provinsi ORDER BY jumlah DESC) AS rank_layanan,  
    SUM(jumlah) OVER (PARTITION BY provinsi) AS total_provinsi,  
    LAG(jumlah) OVER (PARTITION BY provinsi ORDER BY jenis_layanan) AS sebelumnya  
FROM ringkasan  
QUALIFY rank_layanan <= 3  
ORDER BY provinsi, rank_layanan
```

`RANK`, `SUM() OVER`, `LAG / LEAD`, dan `QUALIFY` adalah fitur SQL analitik yang sangat umum dipakai di DuckDB.

LAG dan LEAD: Membandingkan dengan Baris Lain

LAG mengambil nilai dari baris **sebelumnya**, LEAD mengambil nilai dari baris **berikutnya**, dalam urutan yang ditentukan ORDER BY . Keduanya tidak mereduksi jumlah baris.

```
SELECT
    bulan,
    jumlah_permohonan,
    LAG(jumlah_permohonan) OVER (ORDER BY bulan) AS bulan_sebelumnya,
    LEAD(jumlah_permohonan) OVER (ORDER BY bulan) AS bulan_berikutnya
FROM bulanan
ORDER BY bulan
```

Hasil:

bulan	jumlah_permohonan	bulan_sebelumnya	bulan_berikutnya
1	120	NULL	150
2	150	120	135
3	135	150	160
4	160	135	NULL

Baris pertama tidak punya bulan_sebelumnya , baris terakhir tidak punya bulan_berikutnya — keduanya bernilai NULL .

Versi Polars: `.shift()`

Ingat materi Pertemuan 2? `LAG / LEAD` di SQL setara dengan `.shift()` di Polars — `shift(1)` seperti `LAG`, `shift(-1)` seperti `LEAD`.

```
df_bulanan.with_columns([
    pl.col("jumlah_permohonan").shift(1).alias("bulan_sebelumnya"),
    pl.col("jumlah_permohonan").shift(-1).alias("bulan_berikutnya"),
])
```

Hasil (sama persis dengan versi SQL):

bulan	jumlah_permohonan	bulan_sebelumnya	bulan_berikutnya
1	120	null	150
2	150	120	135
3	135	150	160
4	160	135	null

`shift(n)` menggeser nilai sejauh `n` baris ke bawah (positif) atau ke atas (negatif). Gunakan `.over("kota")` untuk menggeser per grup, mirip `PARTITION BY` di SQL.

Kuis Checkpoint 2 (Menit 55)

Buka Wayground — Instructor-Paced Mode

3 soal praktikal SQL DuckDB dan Window Functions:

4. Query DuckDB yang tepat untuk menghitung jumlah transaksi per kota adalah...
5. Klausa SQL yang digunakan untuk memfilter hasil agregasi setelah `GROUP BY` adalah...
6. Fungsi window yang tepat untuk menghitung peringkat penjualan setiap produk adalah...

Waktu: 3 menit

Partitioned Parquet

Menyimpan data dalam partisi (hive-style) membuat DuckDB hanya membaca folder yang relevan.

```
df.with_columns(pl.col("tanggal").dt.year().alias("tahun")).write_parquet(
    "layanan_partisi/",
    use_pyarrow=True,
    pyarrow_options={"partition_cols": ["tahun", "provinsi"]},
)
```

```
-- Hanya partisi tahun=2024 dan provinsi=DKI Jakarta yang dibaca
SELECT jenis_layanan, COUNT(*) AS jumlah
FROM 'layanan_partisi/**/*.*parquet'
WHERE tahun = 2024 AND provinsi = 'DKI Jakarta'
GROUP BY jenis_layanan
```

```
layanan_partisi/
├── tahun=2023/provinsi=DKI Jakarta/data.parquet
├── tahun=2024/provinsi=DKI Jakarta/data.parquet
└── tahun=2024/provinsi=Jawa Barat/data.parquet
```

Interoperabilitas DuckDB <-> Polars

Keduanya berbasis **Apache Arrow** sebagai format in-memory, sehingga pertukaran data hampir **zero-copy**.

```
import polars as pl

df_polars = pl.read_parquet("layanan.parquet")

# Polars -> DuckDB: query langsung DataFrame tanpa menyalin data
con.register("layanan_publik", df_polars)
con.execute("""
    SELECT provinsi, COUNT(*) AS jumlah
    FROM layanan_publik
    GROUP BY provinsi
""").df()

# DuckDB -> Polars: .pl() mengonversi hasil query langsung
hasil = con.execute("""
    SELECT jenis_layanan, SUM(biaya) AS total
    FROM layanan_publik GROUP BY jenis_layanan
""").pl() # <class 'polars.dataframe.frame.DataFrame'>
```

Alur umum: agregasi besar di DuckDB → hasil lebih kecil → lanjutkan transformasi di Polars.

Perbandingan Format Data

Format	Orientasi	Kompresi	Query Langsung	Cocok Untuk
CSV	Baris	Tidak ada	Ya (lambat)	Portabilitas, kompatibilitas universal
Parquet	Kolom	Ya (Snappy/Zstd)	Ya (cepat)	Analitik, penyimpanan jangka panjang
Arrow IPC (Feather)	Kolom	Minimal	Ya (sangat cepat)	Transfer in-memory antar proses

```
# Benchmark waktu baca
for fmt, reader, path in [
    ("Parquet", pl.read_parquet, parquet_path),
    ("CSV", pl.read_csv, csv_path),
    ("Arrow IPC", pl.read_ipc, arrow_path),
]:
    start = time.perf_counter()
    _ = reader(path)
    print(fmt, time.perf_counter() - start)
```

Kuis Checkpoint 3 (Menit 85)

Buka Wayground — Instructor-Paced Mode

4 soal sintesis query dan interoperabilitas:

7. Mengapa predicate pushdown penting ketika DuckDB membaca file Parquet berukuran besar?
8. Query `SELECT kota, SUM(total) AS omzet FROM 'data/transaksi.parquet' GROUP BY kota;` memberikan keuntungan berupa...
9. Anda perlu menggabungkan hasil agregasi DuckDB dengan transformasi Polars. Alur yang paling tepat adalah...
10. Untuk tabel transaksi yang terus bertambah setiap hari, strategi penyimpanan yang paling membantu query berdasarkan tanggal adalah...

Waktu: 4 menit

Ringkasan Pertemuan 3

DuckDB adalah mesin SQL analitik in-process yang:

- Meng-query file Parquet, CSV, dan JSON **langsung tanpa import**
- Mengoptimalkan query otomatis dengan **predicate pushdown** dan **column pruning**
- Menyediakan **Window Functions** SQL yang kaya: RANK, SUM OVER, LAG/LEAD, QUALIFY
- Berinteroperasi **zero-copy** dengan Polars melalui Apache Arrow
- Bekerja optimal dengan **Partitioned Parquet** untuk dataset besar

Check-in Milestone 1 Tugas 1: Repository setup dan data acquisition sudah harus berjalan.

Target Check-in: Sebelum Pertemuan 3 berakhir.

Pertemuan Berikutnya: Data Cleaning dan Quality di Skala Besar — profiling, missing values, dan outliers.

Referensi

- DuckDB Documentation: <https://duckdb.org/docs/>
- DuckDB Python API: <https://duckdb.org/docs/api/python/overview>
- Apache Arrow Project: <https://arrow.apache.org/>

Tautan Kuis

join my quiz.com • 64921709



Kuis sudah termasuk presensi.